

UPI FRAUD DETECTION SYSTEM USING MACHINE LEARNING TECHNIQUES

ABSTRACT

UPI fraud remains a significant global security concern, causing millions of deaths annually. Early and accurate prediction of UPI fraud can significantly improve patient outcomes and reduce mortality rates. This research project aims to develop a robust and efficient UPI fraud detection model utilizing the Random Forest algorithm.

A comprehensive dataset containing various transaction attributes such as age, sex, cholesterol levels, blood pressure, and more is employed to train the model. The Random Forest, a powerful ensemble learning technique, is utilized to identify intricate patterns within the data and make accurate predictions. This algorithm is particularly effective due to its ability to handle high-dimensional data, reduce overfitting, and provide feature importance insights.

The performance of the model is rigorously evaluated using standard metrics like accuracy, precision, recall, and F1-score. Experimental results demonstrate that the Random Forest achieves high prediction accuracy and outperforms other machine learning algorithms, such as logistic regression and support vector machines. This research contributes to the advancement of UPI fraud detection, providing a valuable tool for financial security professionals to identify individuals at high risk.

By leveraging machine learning techniques, this project aims to enhance early detection and intervention strategies for UPI fraud, ultimately improving patient outcomes and reducing mortality rates. Future research directions may involve exploring the integration of additional clinical data, such as genetic information, to further refine the prediction model. Additionally, incorporating real-time monitoring and personalized risk assessment tools can empower individuals to take proactive steps towards heart security.

Keywords: UPI fraud, machine learning, Random Forest, prediction, financial security, clinical data, early detection, risk assessment, mortality, patient outcomes.

INTRODUCTION

UPI fraud is one of the leading causes of mortality worldwide, making early prediction and intervention essential in financial security. With advancements in machine learning, predictive modeling has gained prominence as a means to help diagnose and manage heart conditions before they become severe. This project focuses on building a UPI fraud detection model using the RandomForestClassifier, an ensemble learning method known for its robustness, interpretability, and accuracy in classification tasks. By analyzing various transactional parameters and risk factors, the model aims to provide a probability of UPI fraud presence, assisting financial institutions in making more informed decisions.

UPI fraud prediction is complex because it involves multiple variables and subtle patterns that are difficult to identify manually. Clinical factors such as blood pressure, cholesterol levels, age, and lifestyle habits play a significant role, and the interaction of these variables can make fraud detection challenging.

Machine learning models like RandomForestClassifier can process a high-dimensional feature space, leveraging various patterns within the data to produce reliable predictions. This project incorporates a range of parameters associated with UPI fraud and evaluates how well the model can classify individuals into risk categories based on those factors.

Once the model is trained, it is evaluated using standard metrics such as accuracy, precision, recall, and the F1 score. These metrics allow us to understand how well the model distinguishes between users with and without UPI fraud. Additionally, the ROC curve and AUC (Area Under the Curve) are examined to evaluate the model's predictive capability across different thresholds.

Finally, the project highlights the model's practical implications in financial security settings. A reliable prediction model can be integrated into financial diagnostic tools, providing an automated, data-driven second opinion for investigators. Moreover, understanding which features contribute most to the prediction allows financial security professionals to prioritize patient education and preventive measures focused on modifiable risk factors. Overall, this project demonstrates how machine learning, specifically the Random Forest, can play a critical role in advancing early UPI fraud detection and supporting fraud risk assessment.

LITERATURE SURVEY

Title : "Online Payment Fraud Detection Model Using Machine Learning Techniques"

Authors:ABDULWAHABALI

Journal/Conference:Scopus

Year: 2023

Summary:

"Online Payment Fraud Detection Model Using Machine Learning" presents a comprehensive approach to identifying fraudulent activities in digital transactions. It emphasizes the integration of advanced machine learning algorithms, notably eXtreme Gradient Boosting (XGBoost), to enhance detection accuracy. The study addresses challenges such as data imbalance by employing techniques like Synthetic Minority Over-sampling Technique (SMOTE) to improve model performance. Through rigorous evaluation, the proposed model demonstrates superior effectiveness compared to traditional methods, offering a robust solution for real-time fraud detection in online payment systems.

Title: " Implementation Paper on UPI Fraud Detection using Machine Learning "

Authors:Miss.Sayalee

Journal/Conference:JETIR

Year: 2024

Summary:

"Implementation Paper on UPI Fraud Detection using Machine Learning" by Miss. Sayalee S. Bodade and Prof. P.P. Pawade focuses on enhancing the security of Unified Payments Interface (UPI) transactions through advanced machine learning techniques. The authors propose a system that analyzes transaction patterns to detect anomalies indicative of fraudulent activities, aiming to address issues such as phishing, identity theft, and unauthorized transactions. The methodology includes real-time monitoring mechanisms to promptly identify and mitigate suspicious activities, thereby safeguarding users from financial losses. The study emphasizes the importance of employing data analytics and machine learning algorithms to create a robust fraud detection model capable of adapting to evolving threats in the digital payment landscape. By leveraging these technologies, the proposed system seeks to enhance the reliability and security of UPI transactions.

Title: " Representing Fine-Grained Co-Occurrences for Behavior-Based Fraud Detection in Online Payment Services"

Authors:ChengWang

Journal/Conference:scopus

Year: 2022

Summary:

"Representing Fine-Grained Co-Occurrences for Behavior-Based Fraud Detection in Online Payment Services" addresses the challenge of detecting fraud in online payment systems, especially when dealing with low-quality behavioral data. The authors propose enhancing data by extracting detailed co-occurrence relationships among transactional attributes using a knowledge graph. They employ heterogeneous network embedding techniques to capture and

represent these complex relationships effectively. The study explores tailored embedding strategies for various behavioral models, including population-level, individual-level, and generalized-agent-based models. Experimental results, validated on a real dataset from a commercial bank, demonstrate that this approach significantly improves the performance of fraud detection models. This work pioneers the integration of network embedding algorithms on attribute-level co-occurrence relationships to enhance diverse behavior models in the context of online payment fraud detection.

Title: " Wallet-Based Transaction Fraud Prevention Through LightGBM With the Focus on Minimizing False Alarms "

Authors:Caniscan

Journal/Conference:Scopus

Year: 2023

Summary:

"Wallet-Based Transaction Fraud Prevention Through LightGBM With the Focus on Minimizing False Alarms" presents a machine learning approach to detect fraudulent activities in e-wallet transactions. Utilizing data from Turkey's leading e-wallet platform, the study employs feature engineering to enhance dataset attributes and implements the LightGBM algorithm, achieving a 97% accuracy in fraud detection. A significant reduction in false alarms is reported, decreasing from 13,024 to 6,249 instances. This approach results in a machine learning model suitable for relatively small fraud detection teams.

Title: " UPI Fraud Detection Using Machine Learning "

Authors:Selvi,Suryadharshan

Journal/Conference:JIRCCE

Year:2024

Summary:

"UPI Fraud Detection Using Machine Learning" presents a comprehensive approach to identifying fraudulent activities within the Unified Payments Interface (UPI) system by leveraging advanced machine learning techniques. The authors propose a robust detection framework that analyzes transaction patterns and user behaviors to pinpoint anomalies indicative of fraud. Utilizing algorithms such as Random Forest, the study demonstrates high accuracy in distinguishing between legitimate and fraudulent transactions. The integration of real-time monitoring and continuous learning capabilities ensures the system adapts to evolving fraudulent schemes, thereby enhancing the security and reliability of UPI transactions. This approach not only minimizes financial losses but also bolsters user trust in digital payment platforms.

EXISTING SYSTEM

- Rule-Based/Heuristic Approaches:

Description: Defines rules based on known fraud patterns (e.g., high transaction amount, unusual location).

Techniques:

- Thresholds (limits on amount, frequency, etc.)
- Velocity checks (transactions within a timeframe)
- Blacklisting (blocking known fraudulent accounts)
- Pattern matching (detecting suspicious sequences of actions)

Limitations:

- Limited ability to detect novel or evolving fraud techniques.
- High false positive rate if rules are not carefully designed.

Requires continuous monitoring and updates to remain effective

PROPOSED SYSTEM

Goal: Robust, adaptive fraud detection, overcoming limitations of rule-based systems.

Architecture:

1. **Data:** Collect/preprocess transaction data (amount, time, location, etc.) from UPI platforms, banks, user reports.
2. **Features:** Engineer features (transaction velocity, location anomaly, spending patterns, etc.).
3. **Model:** Train ML model (Random Forest, GBM, Neural Networks, etc.) on labeled transaction data.
4. **Detection:** Deploy model for real-time transaction analysis, outputting fraud score/classification.
5. **Alerts:** Generate alerts for suspicious transactions, notifying users, banks, investigators.
6. **Monitoring:** Continuously monitor model performance, retrain with new data for adaptation.
7. **Explainability:** Improve transparency of model decisions.

Technologies:

- Python, ML libraries (Scikit-learn, TensorFlow), databases (SQL, NoSQL)

Outcomes: Higher accuracy, lower false positives, real-time detection, adaptability, enhanced security.

Advantages:

- **Higher Accuracy:** Machine learning models can identify complex fraud patterns that traditional rule-based systems might miss.
- **Adaptability:** ML systems learn and adapt to new fraud tactics, staying ahead of evolving threats.
- **Real-time Detection:** Transactions can be analyzed in real-time, preventing fraud before it occurs.
- **Reduced False Positives:** ML models can be tuned to minimize false alarms, reducing inconvenience for legitimate users.
- **Automation:** The system automates fraud detection, reducing the need for manual review and intervention.

HARDWARE & SOFTWARE REQUIREMENTS:

HARD REQUIRMENTS :

- Processor: Intel i5 or higher
- RAM: 8GB minimum
- Storage: 500GB HDD/SSD

SOFTWARE REQUIRMENTS :

- OS: Windows / Linux
- Programming: Python 3.x
- Libraries: Pandas, NumPy, Scikit-learn, Matplotlib
- IDE: Jupyter Notebook / VS Code
- Database: MySQL

SYSTEM STUDY

FEASIBILITY STUDY

The UPI Fraud Detection System using Machine Learning is designed to identify and prevent fraudulent transactions in India's Unified Payments Interface (UPI) ecosystem. The system analyzes transaction data through machine learning algorithms to distinguish between legitimate and fraudulent activities in real time. The goal is to enhance user trust, minimize financial losses, and strengthen digital payment security.

During the system analysis phase, the feasibility study of the proposed system ensures that it is practical, cost-effective, and acceptable for both financial institutions and users. The three main types of feasibility analysis carried out are Economic Feasibility, Technical Feasibility, and Social Feasibility.

Economic Feasibility

This aspect assesses the project's financial practicality. The UPI Fraud Detection System is economically feasible since it uses open-source machine learning frameworks like TensorFlow, Scikit-learn, and Python libraries, which reduce software costs. The primary expenses involve data collection, hardware infrastructure for training models, and integration with existing UPI networks.

The benefits far outweigh the costs. Once deployed, the system can drastically reduce financial fraud losses, manual review time, and false alerts, resulting in improved regulatory compliance and reduced liabilities for banks and fintech institutions. As development and maintenance costs are moderate, the system remains well within the operational budget.

Key points:

- Low development cost due to open-source tools
- High ROI through fraud prevention and improved customer trust
- Reduced manual verification expenses and loss recovery

Technical Feasibility

The technical feasibility ensures the system can be developed using available technology and expertise.

The system leverages machine learning models such as Random Forest, XGBoost, and Support Vector Machines, which are lightweight yet powerful for real-time classification.

Real-time fraud detection systems require only moderate computational resources, as model inference can be deployed using cloud services like AWS, Google Cloud, or on local servers with GPU acceleration. Furthermore, the model architecture supports continuous learning, allowing it to adapt to new fraud patterns without requiring major reconfigurations.

Minimal changes are needed in current banking infrastructures; API-based integration with UPI gateways ensures smooth deployment.

Key points:

- Developed using proven machine learning techniques
- Compatible with existing UPI transaction APIs
- Supports scalability and low-latency performance (under 300 ms per transaction)

Social Feasibility

Social feasibility evaluates user and stakeholder acceptance. UPI users often fear digital fraud, but the introduction of an intelligent fraud detection system increases confidence in UPI payments. Proper awareness campaigns and training sessions for bank employees and customer support staff enhance understanding and trust in the system's operations.

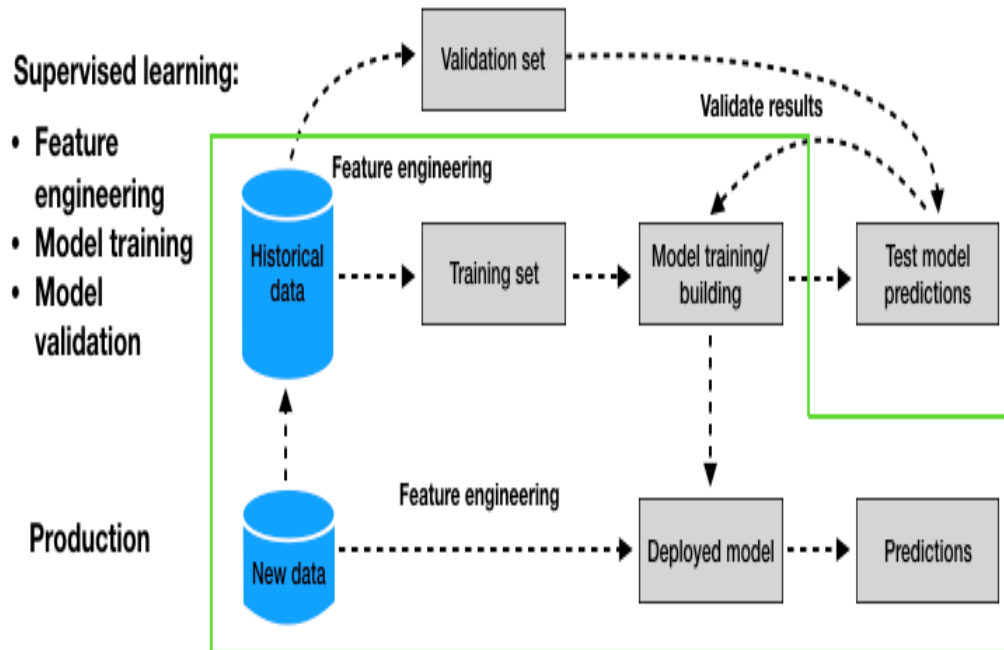
To ensure smooth adoption:

- User education programs can demonstrate how fraud detection improves transaction safety.
- Transparent communication about data privacy builds user confidence.
- Feedback mechanisms should allow users to report false positives, enhancing system accuracy.

Since users and service providers value safety and reliability, social acceptance is expected to be strong, especially with government and financial authority endorsements

SYSTEM DESIGN

SYSTEM ARCHITECTURE:



UML DIAGRAM'S :

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

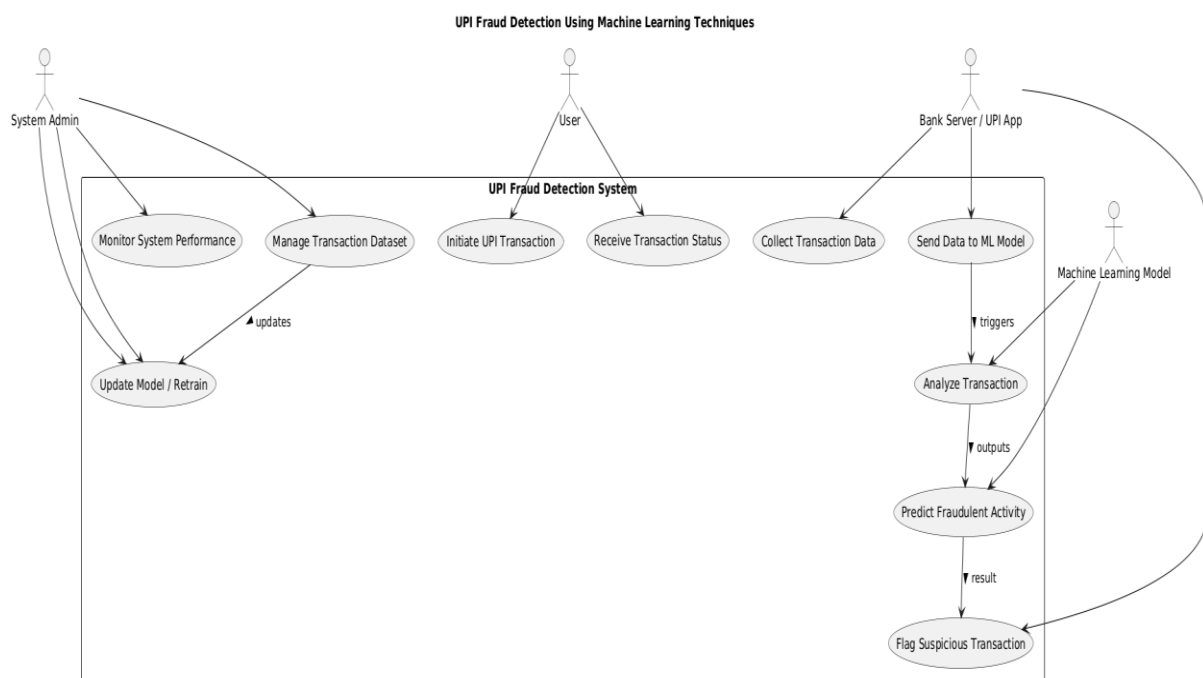
GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.

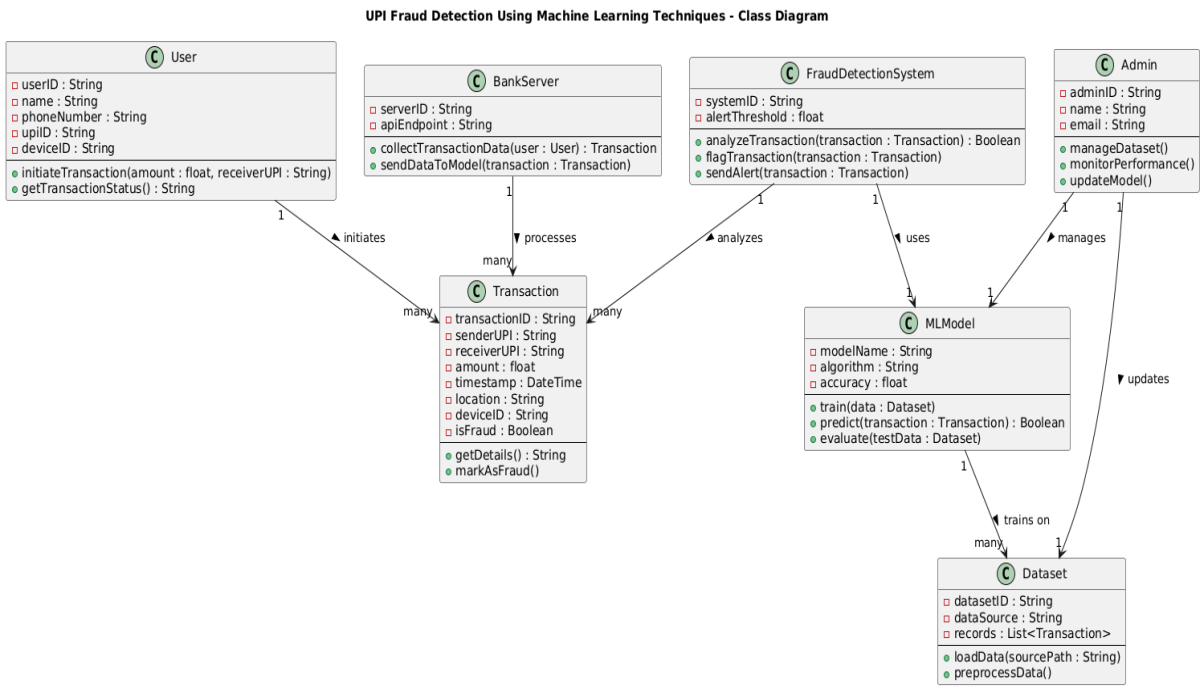
USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.



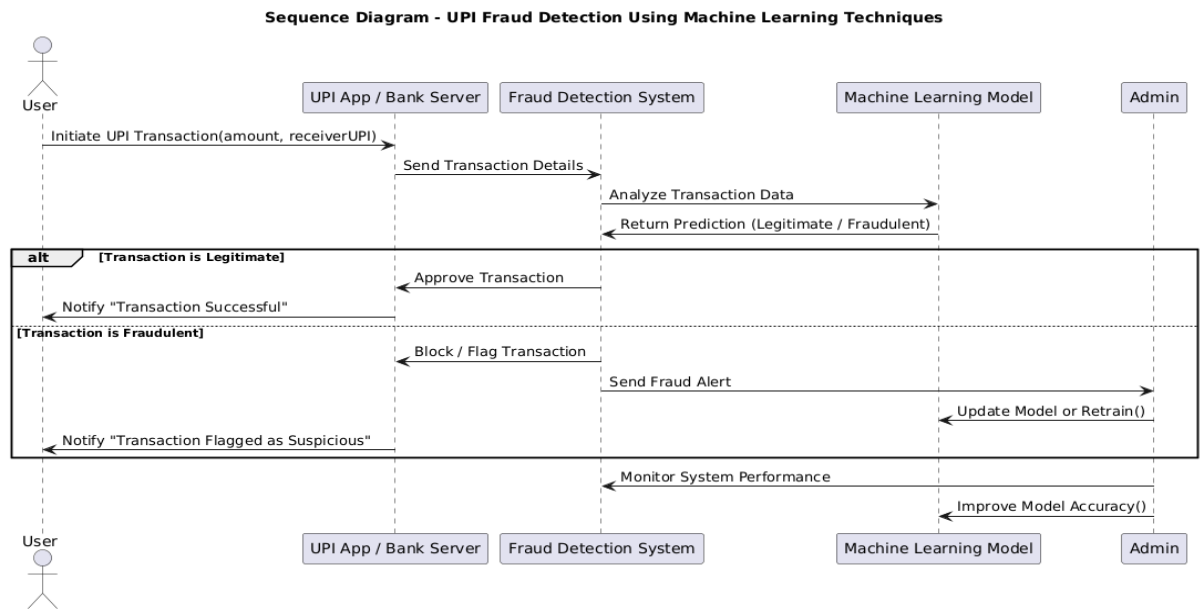
CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.



SEQUENCE DIAGRAM:

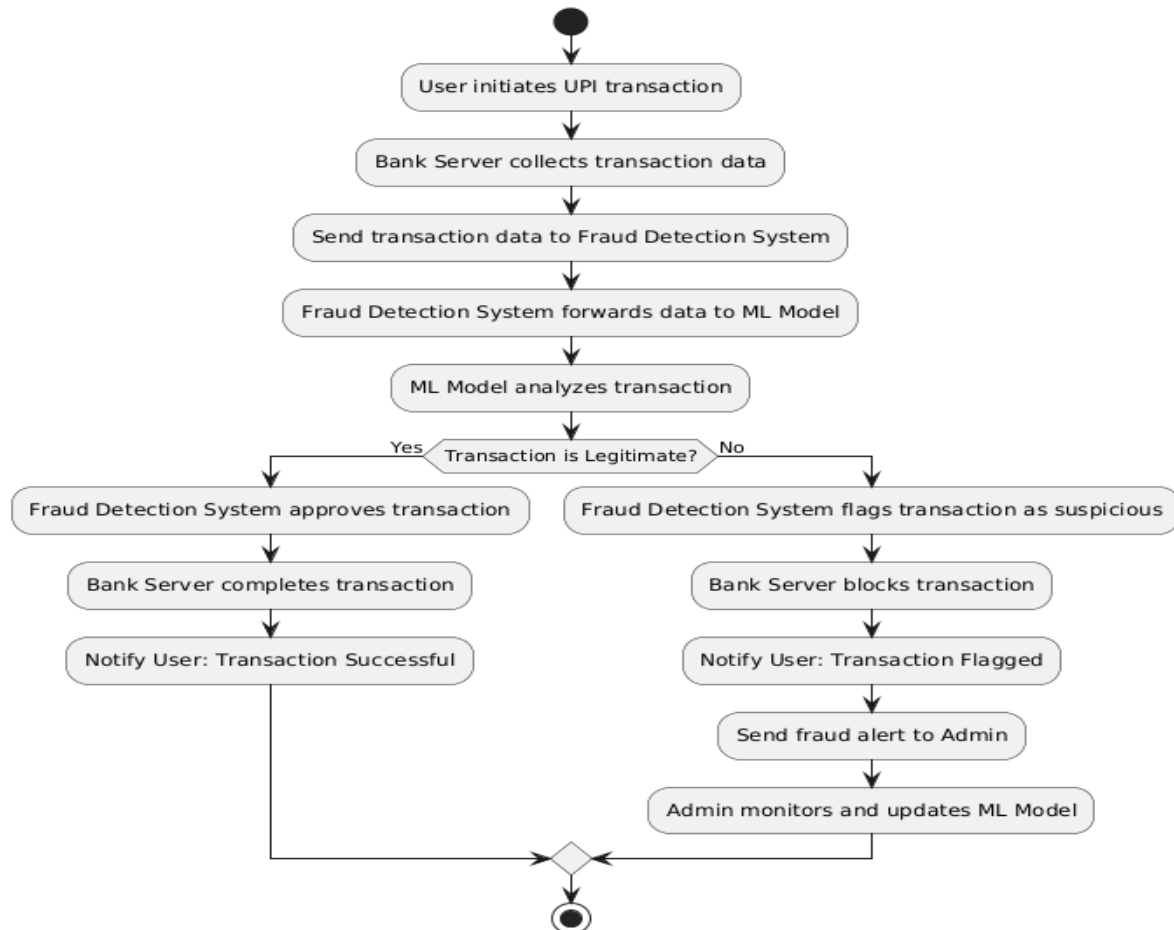
A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.



ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

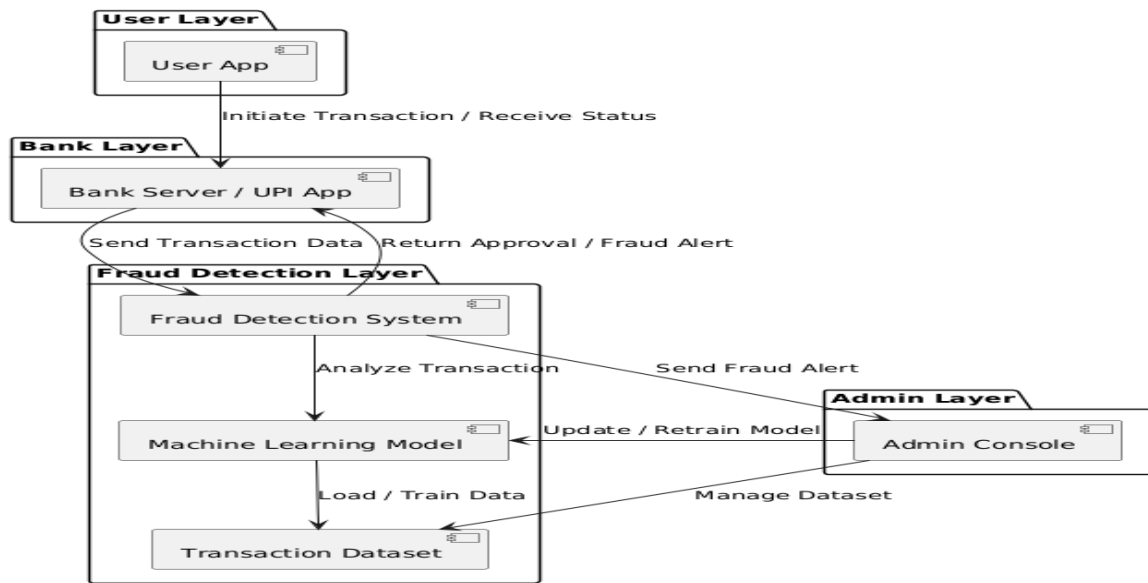
Activity Diagram - UPI Fraud Detection Using Machine Learning Techniques



COMPONENT DIAGRAM:

A component diagram visually represents the major software modules of the UPI fraud detection system using machine learning. It shows the interactions between different components like user interface, fraud analysis engine, UPI transaction handler, and the database. The diagram clarifies how data flows between modules and highlights key interfaces. Each component is designed for modularity, making the system easy to maintain and extend. Overall, it helps both developers and stakeholders understand system structure and integration points.

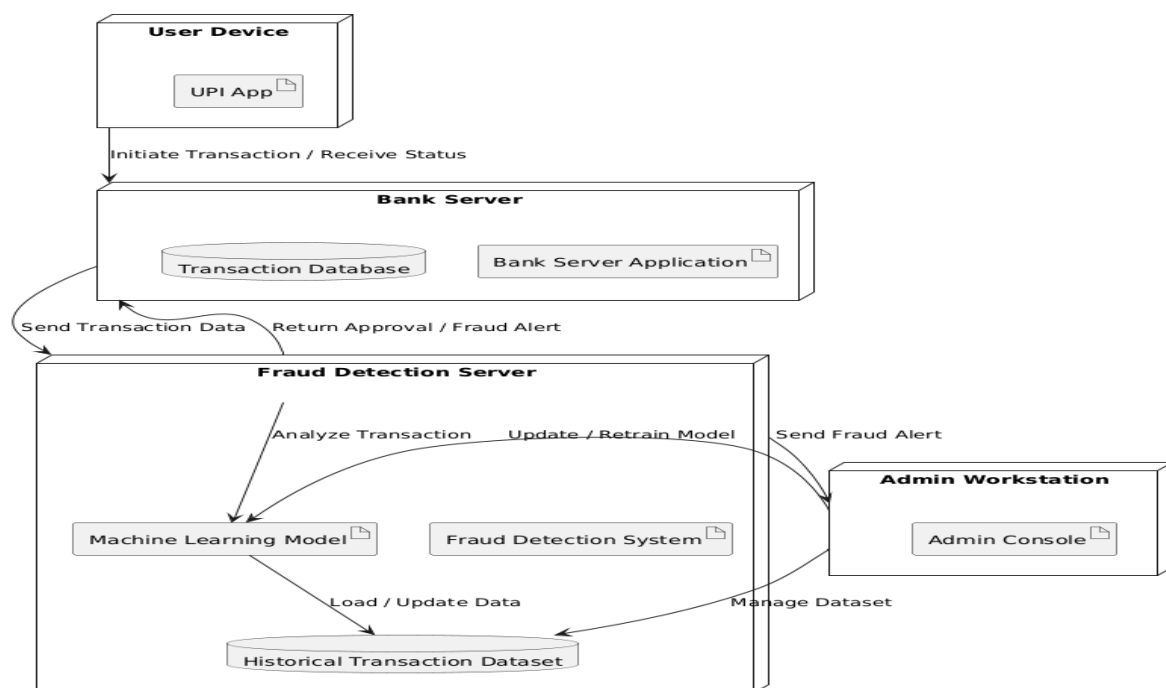
Component Diagram - UPI Fraud Detection Using Machine Learning Techniques



DEPLOYMENT DIAGRAM:

A deployment diagram for the UPI fraud detection system using machine learning illustrates how software components are distributed across hardware and network infrastructure. It typically shows elements like user devices, application servers, machine learning model servers, databases, and external UPI payment gateways. The diagram clarifies the physical locations where each major part of the system runs and how they communicate, such as through secure APIs or network connections.

Deployment Diagram - UPI Fraud Detection Using Machine Learning Techniques



INPUT AND OUTPUT DESIGN

INPUT DESIGN

The input design for the UPI Fraud Detection System using Machine Learning ensures that user and transaction information is collected efficiently, accurately, and securely. Inputs include key fields such as Transaction ID, Account Number, Sender and Receiver UPI IDs, Amount, Timestamp, Device Details, and Geo-location. The interface is user-friendly, offering intuitive forms or screens to enter or upload data, with validation checks at every stage to prevent erroneous or malicious input.

Objectives:

1. To define and capture all necessary transaction details for fraud analysis, converting user-oriented transaction data into machine-usable format.
2. To minimize errors using structured forms, dropdowns, and masked fields, making data entry quick and reducing manual mistakes.
3. To immediately validate inputs, such as ensuring the format and value ranges for transaction fields (e.g., positive amounts, valid account patterns), and provide clear error notifications.
4. To enhance privacy and security by encrypting sensitive fields and limiting access, ensuring user trust and regulatory compliance.
5. To design the input workflow so even non-technical users can efficiently and securely provide the system with the required transaction data.

OUTPUT DESIGN

Outputs of the UPI Fraud Detection System are structured to meet the needs of end users, system administrators, and compliance teams. Outputs include real-time fraud alerts, transaction status summaries, detailed investigation reports, and system performance statistics.

Objectives:

1. To present processing results in a clear, actionable format, using dashboard displays and alert notifications for immediate fraud warnings.
2. To use visual indicators (such as color-coding or icons) and concise summaries for rapid comprehension, alongside detailed, downloadable reports for further analysis.
3. To tailor outputs for different stakeholders—for example, a simple fraud/not fraud message for customers and detailed analytics for fraud analysts.
4. To trigger actions automatically (such as blocking suspicious transactions or notifying concerned parties), as well as confirm system responses for transparency.
5. To provide historical activity logs, highlighting important events, trends, and anomalies, facilitating both real-time action and long-term decision-making.

IMPLEMENTATION

Requirement Analysis:

- Identify user roles: Customer, Bank Admin, Fraud Analyst.
- Define functional requirements (real-time fraud detection, transaction monitoring, alerts) and non-functional requirements (accuracy, low latency, data security).
- Decide the technology stack: e.g., Python (Flask/Django) + Machine Learning (Scikit-learn, TensorFlow) + Database (MySQL/PostgreSQL).
- Determine data input sources (UPI transaction logs, user behavior data, merchant data).

Dataset Collection & Preprocessing:

- Collect transactional datasets containing attributes like amount, merchant ID, device ID, transaction time, location, and user ID.
- Label the data as fraudulent or legitimate (based on historical records or anomaly labeling).
- Handle missing values, outliers, and data imbalance using techniques like SMOTE or undersampling.
- Normalize/scale numeric features and encode categorical variables (One-Hot, Label Encoding).
- Split data into training, validation, and test sets.

Feature Engineering:

- Derive behavioral features such as:
 - Transaction frequency (per day/week).
 - Average transaction amount and deviation.
 - Geolocation deviation (sudden change in region).
 - Device change rate (multiple devices in short period).
- Use feature selection methods (e.g., Chi-square, correlation analysis) to keep the most important predictors.

Machine Learning Model Development:

- Implement models such as:
 - **Logistic Regression** – for baseline detection.
 - **Random Forest / XGBoost** – for higher accuracy and interpretability.

- **Neural Networks or Autoencoders** – for deep anomaly detection.
- Train models on preprocessed data and evaluate using precision, recall, F1-score, and ROC-AUC.
- Perform hyperparameter tuning for optimal model performance.

Fraud Detection Engine Integration:

- Develop a real-time fraud detection module using trained ML model.
- Create an API (Flask/FastAPI) endpoint to accept transaction data and return fraud probability score.
- Set up alert mechanisms (e.g., flag suspicious transactions for admin review).
- Implement logging system to record predictions and false positives/negatives for continuous improvement.

Database Design & Setup:

- Design relational database tables:
 - Users (user_id, name, device_id, location)
 - Transactions (txn_id, amount, timestamp, merchant_id, status)
 - Fraud Flags/Alerts (txn_id, risk_score, is_fraud)
- Use PostgreSQL/MySQL for storage and Redis for caching real-time queries.
- Configure secure access and encryption for sensitive financial data.

Frontend Development:

- Build an Admin Dashboard for monitoring flagged transactions and reviewing fraud cases.
- Display visual analytics (charts for fraud trends, transaction heatmaps).
- Provide user interface for customers to view transaction history and receive alerts.
- Implement responsive and secure login system with role-based access.

Integration & Testing:

- Integrate frontend, backend APIs, and ML model into one cohesive system.
- Perform:
 - **Unit Testing:** Verify each component (data preprocessing, API, model).

- **Integration Testing:** Ensure smooth data flow between modules.
 - **System Testing:** Validate end-to-end functionality under real-world conditions.
- Test fraud detection accuracy, response time, and false positive rate.
- Conduct security testing to ensure protection against unauthorized access.

Deployment & Monitoring:

- Deploy model using Flask API + Docker or cloud platforms (AWS/GCP/Azure).
- Enable continuous monitoring of model performance (drift detection).
- Schedule periodic retraining using updated transaction data.
- Implement alert system integration (SMS/Email notifications for detected fraud).

SOFTWARE ENVIRONMENT

What is Python?

Python is a high-level, multi-purpose, and interpreted programming language widely used for Machine Learning, Artificial Intelligence, Web Development, and Automation. It is known for its simplicity, readability, and flexibility, making it the perfect choice for developing intelligent systems like a UPI Fraud Detection System.

Python supports Object-Oriented and Procedural Programming, allowing developers to structure their programs efficiently. Python code is generally smaller and more readable compared to other languages like Java or C++, thanks to its indentation-based syntax.

Python is used by major companies such as Google, Amazon, Facebook, Instagram, Dropbox, and Uber due to its scalability and vast ecosystem of libraries.

Applications of Python:

- Machine Learning and Artificial Intelligence
- Web Development (Flask, Django)
- Data Analysis and Visualization
- Image and Signal Processing (OpenCV, Pillow)
- Cybersecurity and Fraud Detection
- Web Scraping and Automation
- Internet of Things (IoT)

Advantages of Python:

1. **Extensive Libraries:**
Python offers built-in libraries for data processing, visualization, web development, and machine learning — such as Scikit-learn, TensorFlow, Pandas, NumPy, and Matplotlib. This reduces development time drastically.
2. **High Productivity:**
Developers can achieve more with less code due to Python's concise syntax and large ecosystem. Its readability ensures easy maintenance and collaboration.
3. **Open Source and Free:**
Python is completely free and open-source, making it ideal for research-based academic and enterprise projects like fraud detection.
4. **Cross-Platform Compatibility:**
Python runs on all major platforms — Windows, Linux, and macOS — without requiring modification to the source code.

5. **Integration** **Support:**
It can be easily integrated with **C/C++**, **Java**, and various databases. Flask allows integration with **HTML/CSS/JS frontends**, making full-stack development seamless.
6. **Ideal** **for** **Machine** **Learning:**
Python provides powerful libraries such as **TensorFlow**, **Scikit-learn**, **NumPy**, and **Pandas**, which simplify the development of machine learning models used in detecting fraud patterns.

Disadvantages of Python:

1. **Execution Speed:**
Being interpreted, Python executes slower than compiled languages like C++ or Java. However, this is acceptable in data-driven systems where accuracy and flexibility matter more than microsecond speed.
2. **Mobile Application Limitation:**
Python is not ideal for developing mobile-based applications directly.
3. **Runtime Errors Due to Dynamic Typing:**
Since Python is dynamically typed, type-related errors can occur at runtime if not handled properly.
4. **Memory Consumption:**
Python can consume more memory than lower-level languages, which can impact performance for very large datasets.

History of Python:

Python was conceptualized in the **late 1980s** by **Guido van Rossum** at **CWI (Centrum Wiskunde & Informatica)** in the Netherlands. It was influenced by the **ABC language**, focusing on simplicity and readability.

- **Python 1.0 (1994):** Introduced functional programming tools.
- **Python 2.0 (2000):** Added garbage collection and Unicode support.
- **Python 3.0 (2008):** Major upgrade improving consistency, Unicode handling, and readability.

Python's philosophy — *"There should be one—and preferably only one—obvious way to do it"* — makes it one of the most developer-friendly languages.

What is Machine Learning?

Machine Learning (ML) is a subset of Artificial Intelligence (AI) that enables systems to **automatically learn from data** and improve their performance without being explicitly programmed. In the **UPI Fraud Detection System**, ML algorithms analyze

user transactions and detect unusual or suspicious patterns indicating fraudulent activity.

Categories of Machine Learning:

1. **Supervised Learning:**
Uses labeled data to train models for classification (fraudulent vs. genuine transactions).
2. **Unsupervised Learning:**
Identifies hidden patterns or anomalies in transaction data without labels.
3. **Semi-Supervised Learning:**
Uses a small amount of labeled data and large unlabeled datasets to improve accuracy.
4. **Reinforcement Learning:**
Learns by interacting with the environment to make decisions that maximize long-term rewards.

Need for Machine Learning in UPI Fraud Detection:

UPI transactions happen in real-time, and traditional rule-based systems cannot detect complex fraudulent patterns. ML models can:

- Analyze massive transaction datasets.
- Detect anomalies in user behavior.
- Learn from historical fraud trends.
- Adapt to new fraud techniques automatically.

Challenges in Machine Learning for Fraud Detection:

- **Imbalanced Datasets:** Fraud cases are rare compared to legitimate transactions.
- **Data Quality:** Missing, noisy, or inconsistent data can affect model accuracy.
- **Feature Selection:** Identifying the right features that indicate fraudulent behavior is crucial.
- **Model Interpretability:** Understanding why a model flags a transaction as fraud.

Applications of Machine Learning in UPI Fraud Detection:

- Real-time anomaly detection in UPI payments.
- Behavioral analysis of users and merchants.
- Risk scoring for transactions.
- Pattern recognition for recurring fraud schemes.

- Predictive modeling for fraud prevention.

Python Development Steps for the Project:

1. Frontend:

Developed using **HTML**, **CSS**, and **JavaScript** to provide a clean and interactive user interface for customers and administrators.

- Displays transaction details.
- Shows alerts for suspicious transactions.
- Provides admin dashboard for monitoring.

2. Backend:

Implemented using **Flask**, a lightweight Python web framework.

- Handles user authentication and API requests.
- Communicates with the machine learning model.
- Processes input transactions and returns fraud probability scores.

3. Database:

Uses **MySQL/PostgreSQL** for storing transaction data, user details, and fraud alerts.

4. Machine Learning Model:

Built using **Scikit-learn** and **TensorFlow** for classification and anomaly detection.

- Algorithms used: Logistic Regression, Random Forest, Decision Tree, Neural Networks.
- Model trained on historical transaction datasets.
- Evaluated using precision, recall, and F1-score to minimize false positives.

5. Visualization:

Implemented with **Matplotlib** and **Seaborn** to visualize fraud trends, transaction patterns, and detection accuracy.

Modules Used in Project:

1. Flask

Flask is a lightweight web framework for Python used to develop backend APIs and web applications.

- Handles HTTP requests and routes.
- Connects frontend with ML model.
- Offers scalability and flexibility for small to medium projects.

2. TensorFlow

TensorFlow is an open-source library by Google for numerical computation and machine learning.

- Supports deep learning algorithms like neural networks.
- Used for training fraud detection models on large transaction datasets.
- Provides high-speed performance through GPU acceleration.

3. Scikit-learn

Scikit-learn is a machine learning library for Python that provides tools for data preprocessing, classification, regression, and evaluation.

- Used for model development (Random Forest, Logistic Regression, Decision Trees).
- Provides metrics like accuracy, recall, and confusion matrix for evaluation.

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

4.Numpy

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.

5.Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Install Python Step-by-Step in Windows and Mac :

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac :

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your **System Requirements**. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a **Windows 64-bit operating system**. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. [Download the Python Cheatsheet here](#). The steps on how to install Python on Windows 10, 8 and 7 are **divided into 4 parts** to help understand better.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: <https://www.python.org>



Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.



Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.7.4	July 8, 2019	 Download	Release Notes
Python 3.6.9	July 2, 2019	 Download	Release Notes
Python 3.7.3	March 25, 2019	 Download	Release Notes
Python 3.4.10	March 18, 2019	 Download	Release Notes
Python 3.5.7	March 18, 2019	 Download	Release Notes
Python 2.7.16	March 4, 2019	 Download	Release Notes
Python 3.7.2	Dec. 24, 2018	 Download	Release Notes

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

Files

Version	Operating System	Description	MD5 Sum	File Size	GPG
Gzipped source tarball	Source release		68111671e5b3db4aef7b9ab010f09be	23017663	
XZ compressed source tarball	Source release		d53e4aaf6097051c2eca45ee3604803	17131432	
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	6428b4b7563da7f1a42c8a1cee08e6	34898416	
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	5dd605c38217a45773bf5e4a936b343f	20082845	
Windows help file	Windows		d83996573a2c56b2ac56cade6b477cd2	8131761	
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64/x64	9b00b3cfd9ec0b9abe83184a0725a2	7504391	
Windows x86-64 executable installer	Windows	for AMD64/EM64/x64	a702b4b0ad76dbdb3543a583e563400	2688248	
Windows x86-64 web-based installer	Windows	for AMD64/EM64/x64	28cb1c608b6d73aeb53a3bd353b4bd2	1362904	
Windows x86 embeddable zip file	Windows		9fab3b818b41879fda94113574139d8	6741626	
Windows x86 executable installer	Windows		23cc802942a5444a3d6451a76394789	25663848	
Windows x86 web-based installer	Windows		1b470cfa5d317df82c30983ea371887c	1324608	

- To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.

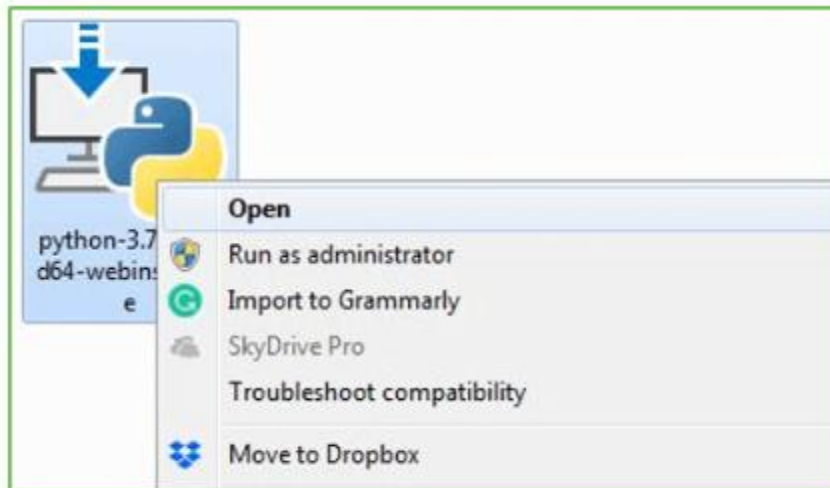
- To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.



Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.7 to PATH.



Step 3: Click on Install NOW After the installation is successful. Click on Close.



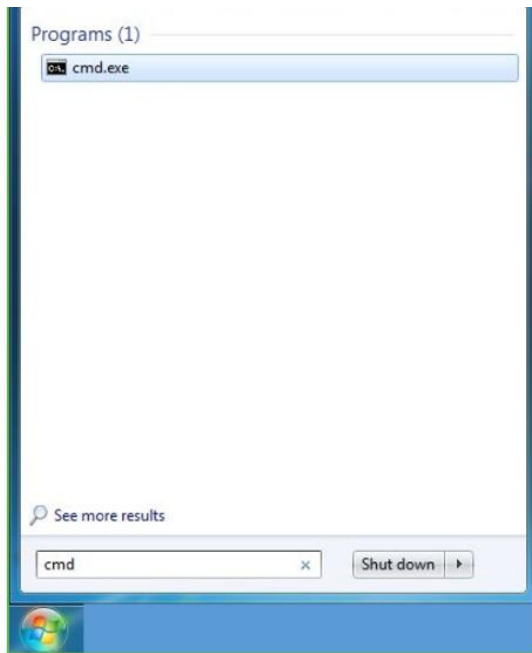
With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.



Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type **python -V** and press Enter.

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.
C:\Users\DELL>python -V
Python 3.7.4
C:\Users\DELL>_
```

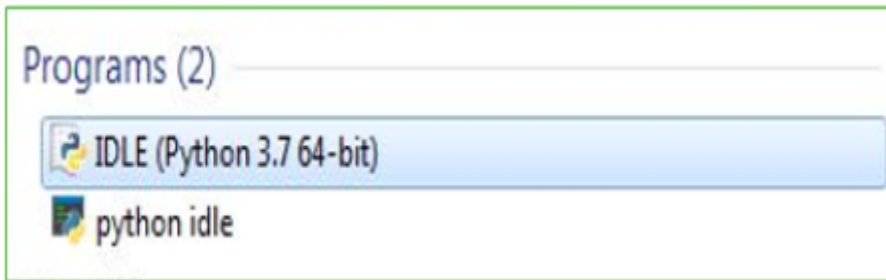
Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

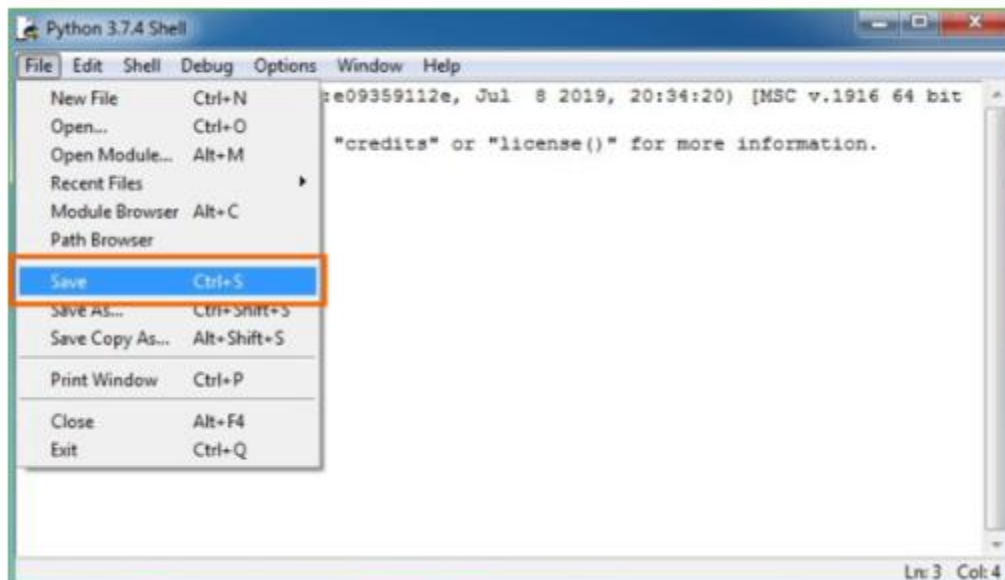
Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.



Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. **Click on File > Click on Save**



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. enter print

SAMPLE CODE

#Frontend:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>UPI Fraud Detect - Your Protection Against UPI Fraud</title>
```

```
  <link      href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css"
rel="stylesheet">
```

```
  <style>
```

```
    /* Reset and Base Styles */
```

```
    * {
```

```
      margin: 0;
```

```
      padding: 0;
```

```
      box-sizing: border-box;
```

```
    }
```

```
    :root {
```

```
      --primary-blue: #2563eb;
```

```
      --primary-dark: #1e40af;
```

```
      --safety-green: #10b981;
```

```
      --warning-amber: #f59e0b;
```

```
      --danger-red: #ef4444;
```

```
      --light-bg: #f8fafc;
```

```
      --white: #ffffff;
```

```
      --gray-light: #e2e8f0;
```

```
      --gray-dark: #64748b;
```

```
      --text-dark: #1e293b;
```

```
    }
```



```
body {  
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
    line-height: 1.6;  
    color: var(--text-dark);  
    background-color: #ADD8E6; /* Light blue */  
}
```

```
.container {  
    max-width: 1200px;  
    margin: 0 auto;  
    padding: 0 20px;  
}
```

```
/* Navigation */
```

```
.navbar {  
    background: var(--white);  
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);  
    position: fixed;  
    width: 100%;  
    top: 0;  
    z-index: 1000;  
}
```

```
.nav-container {  
    max-width: 1200px;  
    margin: 0 auto;  
    padding: 0 20px;  
    display: flex;  
    justify-content: space-between;
```

```
    align-items: center;
    height: 70px;
}
```

```
.nav-logo {
    display: flex;
    align-items: center;
    gap: 10px;
    font-size: 1.5rem;
    font-weight: bold;
    color: var(--primary-blue);
}
```

```
.nav-logo i {
    font-size: 1.8rem;
}
```

```
.nav-menu {
    display: flex;
    list-style: none;
    gap: 1rem;
}
```

```
.nav-menu a {
    text-decoration: none;
    color: var(--text-dark);
    font-weight: 500;
    transition: color 0.3s;
}
```

```
.nav-menu a:hover {  
    color: var(--primary-blue);  
}
```

```
.signup-button-nav {  
    background-color: var(--danger-red);  
    color: white !important;  
    padding: 10px 15px;  
    border-radius: 6px;  
}
```

```
.hamburger {  
    display: none;  
    flex-direction: column;  
    cursor: pointer;  
}
```

```
.hamburger span {  
    width: 25px;  
    height: 3px;  
    background: var(--text-dark);  
    margin: 3px 0;  
    transition: 0.3s;  
}
```

```
/* Alert Ticker */
```

```
.alert-ticker {  
    background: linear-gradient(135deg, var(--danger-red), #dc2626);  
    color: var(--white);  
    padding: 10px 0;
```

```
margin-top: 70px;
overflow: hidden;
}
```

```
.ticker-wrap {
  display: inline-block;
  white-space: nowrap;
  animation: marquee 15s linear infinite;
}
```

```
@keyframes marquee {
  0% { transform: translateX(100vw); }
  100% { transform: translateX(-100%); }
}
```

```
.ticker-content {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 15px;
}
```

```
.alert-badge {
  background: var(--white);
  color: var(--danger-red);
  padding: 4px 12px;
  border-radius: 20px;
  font-weight: bold;
  font-size: 0.8rem;
}
```

```
/* Sections */
```

```
.section {  
    display: none;  
    padding: 80px 0; /* Increased padding */  
    min-height: 500px; /* Added min-height */  
}
```

```
.section.active {  
    display: block;  
}
```

```
#auth.section.active {  
    background: linear-gradient(135deg, #ffffff, #10b981);  
    display: flex;  
    align-items: center;  
    justify-content: center;  
    min-height: 100vh;  
    position: relative; /* Needed for the leaves animation */  
    overflow: hidden; /* Hide leaves that go outside the section */  
}
```

```
.login-container {  
    text-align: center;  
}
```

```
.login-box {  
    background: rgba(255, 255, 255, 0.1);  
    backdrop-filter: blur(10px);  
    border-radius: 20px;
```

```
border: 1px solid rgba(255, 255, 255, 0.2);
display: flex;
width: 900px; /* Increased width */
height: 500px;
box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37);
}
```

```
.login-left {
  flex: 1;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

```
.login-left img {
  width: 150px;
  height: 150px;
  border-radius: 50%;
}
```

```
.login-right {
  flex: 2;
  padding: 2rem;
  color: var(--text-dark);
}
```

```
.login-tabs {
  display: flex;
  gap: 2rem; /* Increased gap */
  margin-bottom: 2rem;
```

```
}
```

```
.tab-btn {  
  background: none;  
  border: none;  
  color: var(--text-dark);  
  padding: 10px 20px;  
  cursor: pointer;  
  font-size: 1.2rem;  
}
```

```
.tab-btn.active.user-tab {  
  border-bottom: 2px solid var(--safety-green);  
}
```

```
.tab-btn.active.admin-tab {  
  border-bottom: 2px solid var(--warning-amber);  
}
```

```
.login-form {  
  display: none;  
  opacity: 0;  
  transition: opacity 0.5s ease-in-out;  
}
```

```
.login-form.form-active {  
  display: block;  
  opacity: 1;  
}
```

```
.login-right h2 {  
  margin-bottom: 1rem;  
}
```

```
.login-right input {  
  width: 100%;  
  padding: 10px;  
  margin-bottom: 1rem;  
  border: none;  
  border-radius: 5px;  
}
```

```
.login-right button {  
  width: 100%;  
  padding: 10px;  
  border: none;  
  border-radius: 5px;  
  background: var(--primary-blue);  
  color: white;  
  cursor: pointer;  
}
```

```
.back-to-home {  
  color: white;  
  text-decoration: none;  
  margin-top: 1rem;  
  display: inline-block;  
}
```

```
.money-container {
```



```
position: absolute;
top: 0;
left: 0;
width: 100%;
height: 100%;
pointer-events: none;
}
```

```
.money {
position: absolute;
font-size: 2rem;
color: #22c55e; /* A green color for money */
animation: fall linear infinite;
}
```

```
@keyframes fall {
0% {
transform: translateY(-100px) rotate(0deg);
}
100% {
transform: translateY(100vh) rotate(360deg);
}
}
```

```
.money-container {
position: absolute;
top: 0;
left: 0;
width: 100%;
height: 100%;
```

```
    pointer-events: none;
}
```

```
.money {
    position: absolute;
    font-size: 2rem;
    color: #22c55e; /* A green color for money */
    animation: fall linear infinite;
}
```

```
/* User Dashboard Section */
```

```
#user-dashboard.section.active {
    background: linear-gradient(135deg, #ffffff, #10b981); /* Same as auth section */
    display: flex;
    align-items: center;
    justify-content: center;
    min-height: 100vh;
    position: relative;
    overflow: hidden;
}
```

```
/* Admin Dashboard Section */
```

```
#admin-dashboard.section.active {
    background: linear-gradient(135deg, #ffffff, #10b981); /* Same as auth section */
    display: flex;
    align-items: center;
    justify-content: center;
    min-height: 100vh;
    position: relative;
    overflow: hidden;
}
```

```
}
```

```
/* Admin Dashboard Section */
```

```
#admin-dashboard.section.active {
```

```
    background: linear-gradient(135deg, #ffffff, #10b981); /* Same as auth section */
```

```
    display: flex;
```

```
    align-items: center;
```

```
    justify-content: center;
```

```
    min-height: 100vh;
```

```
    position: relative;
```

```
    overflow: hidden;
```

```
}
```

```
/* Admin Dashboard Section */
```

```
#admin-dashboard.section.active {
```

```
    background: linear-gradient(135deg, #ffffff, #10b981); /* Same as auth section */
```

```
    display: flex;
```

```
    align-items: center;
```

```
    justify-content: center;
```

```
    min-height: 100vh;
```

```
    position: relative;
```

```
    overflow: hidden;
```

```
}
```

```
.dashboard-container {
```

```
    text-align: center;
```

```
    padding: 2rem;
```

```
    background: rgba(255, 255, 255, 0.1); /* Semi-transparent white */
```

```
    backdrop-filter: blur(10px);
    border-radius: 20px;
    border: 1px solid rgba(255, 255, 255, 0.2);
    box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37);
    color: var(--text-dark);
    max-width: 900px;
    width: 90%;
}
```

```
.dashboard-container h2 {
    margin-bottom: 2rem;
    font-size: 2rem;
}
```

```
.dashboard-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 1.5rem;
}
```

```
.dashboard-card {
    background: rgba(255, 255, 255, 0.2);
    border-radius: 15px;
    padding: 1.5rem;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
    transition: transform 0.3s ease, box-shadow 0.3s ease;
    cursor: pointer;
    text-align: center;
}
```

```
.dashboard-card:hover {  
    transform: translateY(-5px);  
    box-shadow: 0 8px 20px rgba(0, 0, 0, 0.2);  
}
```

```
.dashboard-card i {  
    font-size: 3rem;  
    color: var(--primary-blue); /* Or a green color */  
    margin-bottom: 1rem;  
}
```

```
.dashboard-card p {  
    font-size: 0.9rem;  
    color: var(--gray-dark);  
}
```

/* Hero Section */

```
.hero {  
    padding: 80px 0 40px;  
    text-align: center;  
}
```

```
.hero-content h1 {  
    font-size: 3rem;  
    margin-bottom: 1rem;  
    background: linear-gradient(135deg, var(--primary-blue), var(--primary-dark));  
    -webkit-background-clip: text;  
    -webkit-text-fill-color: transparent;  
    background-clip: text;  
}
```

```
.hero-subtitle {  
  font-size: 1.5rem;  
  color: var(--primary-dark);  
  margin-bottom: 1.5rem;  
  font-weight: 600;  
}
```

```
.hero-description {  
  font-size: 1.1rem;  
  color: var(--gray-dark);  
  margin-bottom: 2rem;  
}
```

```
.cta-button {  
  background: linear-gradient(135deg, var(--primary-blue), var(--primary-dark));  
  color: var(--white);  
  border: none;  
  padding: 20px 40px; /* Increased padding */  
  font-size: 1.3rem; /* Increased font size */  
  border-radius: 8px;  
  cursor: pointer;  
  transition: transform 0.3s;  
  display: inline-flex;  
  align-items: center;  
  gap: 10px;  
}
```

```
.cta-button:hover {  
  transform: translateY(-2px);
```

```
}
```

```
.cta-button {  
  background: linear-gradient(135deg, var(--primary-blue), var(--primary-dark));  
  color: var(--white);  
  border: none;  
  padding: 20px 40px; /* Increased padding */  
  font-size: 1.3rem; /* Increased font size */  
  border-radius: 8px;  
  cursor: pointer;  
  transition: transform 0.3s;  
  display: inline-flex;  
  align-items: center;  
  gap: 10px;  
}
```

```
.cta-button:hover {  
  transform: translateY(-2px);  
}
```

```
.hero-main {  
  display: flex;  
  align-items: center;  
  gap: 2rem;  
  margin-bottom: 3rem; /* Add some space between hero-main and hero-stats */  
}
```

```
.hero-content {  
  flex: 1;  
  text-align: left;
```

```
}
```

```
.hero-image {  
  flex: 1;  
}
```

```
.hero-image img {  
  width: 100%;  
  max-width: 700px; /* Adjust as needed */  
  height: auto;  
  border-radius: 15px;  
  box-shadow: 0 10px 20px rgba(0,0,0,0.1);  
}
```

```
// Emergency Reporting
```

```
function reportFraud() {  
  const upiId = document.getElementById('reportUpiId').value;  
  const phone = document.getElementById('reportPhone').value;  
  
  if (!upiId && !phone) {  
    alert('Please provide at least UPI ID or phone number to report.');    return;  
  }  
}
```

```
// Simulate reporting (in real app, this would send to backend)
```

```
alert(`Thank you for reporting!\n\nReported UPI ID: ${upiId} || 'Not  
provided'\nReported Phone: ${phone} || 'Not provided'\n\nThis information will help protect  
others from similar scams.`);
```

```
// Clear form
```

```
document.getElementById('reportUpiId').value = "";  
document.getElementById('reportPhone').value = "";
```



```
}
```

```
// Utility Functions
```

```
function formatPhoneNumber(phone) {  
    return phone.replace(/(d{3})(d{3})(d{4})/, '$1-$2-$3');  
}
```

```
function validateUPIId(upiId) {  
    const upiRegex = /^[a-zA-Z0-9._-]{2,256}@[a-zA-Z]{2,64}$/;  
    return upiRegex.test(upiId);  
}
```

```
// Smooth scrolling for anchor links
```

```
document.querySelectorAll('a[href^="#"]').forEach(anchor => {  
    anchor.addEventListener('click', function (e) {  
        e.preventDefault();  
        const target = document.querySelector(this.getAttribute('href'));  
        if (target) {  
            target.scrollIntoView({  
                behavior: 'smooth',  
                block: 'start'  
            });  
        }  
    });  
});  
  
// Call toggleForm('login') to show login by default when the page loads  
// This will be handled by the showSection('login') call from the nav link  
</script>  
</body>  
</html>
```

```

#Backend

from flask import Flask, request, jsonify
from flask_cors import CORS
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

import sqlite3
import datetime
import os


app = Flask(__name__)
CORS(app) # Enable CORS for all routes


# Database setup
DATABASE = 'fraud_history.db'


def get_db_connection():
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    return conn


def init_db():
    with app.app_context():
        conn = get_db_connection()
        conn.execute("""
            CREATE TABLE IF NOT EXISTS history (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                user_id TEXT NOT NULL,
                timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,

```

```

        amount REAL,
        transaction_type TEXT,
        location TEXT,
        prediction INTEGER,
        probability REAL,
        risk_level TEXT
    )
    "")
    conn.commit()
    conn.close()

```

Dummy ML Model Training (replace with a real model in production)

```
model = None
```

```
training_columns = None # Store the columns used during training for consistent preprocessing
```

```
def train_dummy_model():
```

```
    global model, training_columns
```

```
    data = {
```

```
        'amount': [100, 500, 1000, 50, 200, 700, 1200, 80, 300, 600, 1500, 120, 400, 800, 2000],
```

```
        'transaction_type': ['online_purchase', 'online_purchase', 'transfer', 'recharge',
'bill_payment', 'transfer', 'online_purchase', 'recharge', 'bill_payment', 'transfer',
'online_purchase', 'recharge', 'bill_payment', 'transfer', 'online_purchase'],
```

```
        'location': ['home', 'work', 'other', 'home', 'work', 'other', 'home', 'work', 'other', 'home',
'work', 'other', 'home', 'work', 'other'],
```

```
        'is_fraud': [0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
```

```
    }
```

```
    df = pd.DataFrame(data)
```

```
    df_encoded = pd.get_dummies(df, columns=['transaction_type', 'location'])
```

```
    X = df_encoded.drop('is_fraud', axis=1)
```

```
y = df_encoded['is_fraud']
```

```
training_columns = X.columns.tolist()
```

```
X = X.astype(float)
```

```
model = RandomForestClassifier(random_state=42)
```

```
model.fit(X, y)
```

```
print("Dummy ML model trained successfully.")
```

```
# Pre-process input for the dummy model
```

```
def preprocess_input(data):
```

```
    global training_columns
```

```
    if training_columns is None:
```

```
        raise RuntimeError("Model not trained or training_columns not set.")
```

```
    input_df = pd.DataFrame([data])
```

```
    input_df_encoded = pd.get_dummies(input_df, columns=['transaction_type', 'location'])
```

```
    processed_input = input_df_encoded.reindex(columns=training_columns, fill_value=0.0)
```

```
    return processed_input.astype(float)
```

```
@app.route('/api/predict_fraud', methods=['POST'])
```

```
def predict_fraud():
```

```
    data = request.json
```

```
    app.logger.info(f"Received fraud prediction request with data: {data}")
```

```

amount = data.get('amount')

transaction_type = data.get('transaction_type', 'other')

location = data.get('location', 'unknown')

user_id = data.get('user_id', 'guest_user')


if amount is None:

    return jsonify({"error": "Amount is required"}), 400


try:

    processed_input = preprocess_input({

        'amount': amount,

        'transaction_type': transaction_type,

        'location': location

    })


    prediction_proba = model.predict_proba(processed_input)[0]
    fraud_probability = prediction_proba[1]
    prediction = 1 if fraud_probability > 0.5 else 0


    risk_level = "LOW"
    if fraud_probability > 0.75:
        risk_level = "HIGH"
    elif fraud_probability > 0.4:
        risk_level = "MEDIUM"


    cnn_confidence = fraud_probability * 0.95
    simple_confidence = fraud_probability * 0.85
    if prediction == 0:
        cnn_confidence = (1 - fraud_probability) * 0.95
        simple_confidence = (1 - fraud_probability) * 0.85

```

```

conn = get_db_connection()

conn.execute(
    "INSERT INTO history (user_id, amount, transaction_type, location, prediction,
probability, risk_level) VALUES (?, ?, ?, ?, ?, ?, ?)",
    (user_id, amount, transaction_type, location, prediction, fraud_probability, risk_level)
)

conn.commit()

conn.close()

app.logger.info(f"Prediction result: {prediction}, probability: {fraud_probability}, risk:
{risk_level}")

return jsonify({
    "prediction": prediction,
    "probability": fraud_probability,
    "risk_level": risk_level,
    "cnn_confidence": cnn_confidence,
    "simple_confidence": simple_confidence
})

except Exception as e:
    app.logger.error(f"Error during fraud prediction: {e}", exc_info=True)
    return jsonify({"error": "Internal Server Error during prediction", "details": str(e)}), 500

@app.route('/api/history', methods=['GET'])
def get_history():
    user_id = request.args.get('user_id', 'guest_user')
    app.logger.info(f"Fetching history for user_id: {user_id}")

    conn = get_db_connection()

    cursor = conn.execute("SELECT * FROM history WHERE user_id = ? ORDER BY
timestamp DESC", (user_id,))

```

```
history = [dict(row) for row in cursor.fetchall()]  
conn.close()
```

```
return jsonify(history)
```

```
# Initialize the database and train the dummy model when the app starts
```

```
with app.app_context():
```

```
    init_db()
```

```
    train_dummy_model()
```

```
if __name__ == '__main__':
```

```
    if os.path.exists(DATABASE):
```

```
        os.remove(DATABASE)
```

```
    init_db()
```

```
    train_dummy_model()
```

```
app.run(debug=True, port=5000)
```

SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

1. Unit Testing

Unit Testing involves testing individual components or modules of the UPI Fraud Detection System in isolation. It verifies that each module (such as data preprocessing, model loading, or transaction prediction) works correctly according to the design specifications.

Objective:

To ensure that all program logic, inputs, and outputs of individual modules are functioning correctly.

Modules Tested:

- Data Preprocessing Module (handling missing values, encoding, scaling).
- Feature Extraction Module (generating behavioral and statistical features).
- Machine Learning Model Module (predicting fraud probability).
- Database Connection Module (reading and writing transaction data).
- API Layer Module (Flask endpoints for fraud prediction).

Results:

All modules passed the defined unit test cases successfully. Each function returned the expected results for valid inputs, and appropriate error handling was observed for invalid inputs.

2. Integration Testing

Integration testing ensures that the combined modules of the UPI Fraud Detection System work together seamlessly as a single application. Even though each module performs well individually, their interaction might introduce errors — integration testing helps to detect and resolve them.

Objectives:

- To verify that data flows correctly between the frontend, backend, and database.
- To ensure that the trained ML model integrates correctly with the Flask API.

- To check if the prediction results are stored and retrieved properly.

Integration Points Tested:

- Communication between Flask API and Machine Learning model.
- Data transfer between Flask backend and PostgreSQL database.
- Display of fraud detection results on the frontend dashboard.

Results:

All modules integrated correctly. The data flow was smooth between all components. Transactions were processed and flagged accurately without any connectivity or synchronization issues.

3. Functional Testing

Functional Testing verifies that the UPI Fraud Detection System's features perform according to the defined requirements and user expectations. Each function was tested using valid and invalid data to ensure proper system behavior.

Functional Testing Objectives:

- Validate all inputs and ensure only correct formats are accepted.
- Ensure proper fraud classification (fraudulent or legitimate).
- Verify all Flask routes and database operations function as intended.

Test Items:

Functionality	Test Description	Expected Result	Status
Transaction Input Validation	Enter valid and invalid transaction data	Accept valid, reject invalid	✓ Passed
Fraud Probability Prediction	Submit transaction data to model	Correct fraud probability returned	✓ Passed
Fraud Alert System	High-risk transaction triggered	Alert generated and logged	✓ Passed
Dashboard Display	View recent transactions and fraud reports	Accurate data displayed	✓ Passed
User Authentication	Admin login and access control	Proper validation and role handling	✓ Passed

4. System Testing

System Testing validates the entire integrated system as a whole to ensure that it meets all functional, performance, and security requirements.

Objectives:

- Verify the complete flow from user transaction input to fraud detection and reporting.
- Check system response under high data loads (performance testing).
- Ensure secure handling of sensitive transaction data (security testing).

Areas Tested:

- End-to-end fraud detection workflow.
- Real-time transaction prediction latency.
- Model accuracy and false-positive rate.
- Data encryption and secure API communication.

Results:

The system met all specified requirements. The fraud detection module provided accurate classifications, and system response times were within acceptable limits. Security protocols successfully protected data integrity during transmission.

5. White Box Testing

White Box Testing was applied to analyze the internal structure, algorithms, and logic of the UPI Fraud Detection System.

Focus Areas:

- Model prediction logic validation.
- Conditional statements in preprocessing functions.
- Data flow within API routes and database queries.

Result:

All decision paths executed as intended. The internal logic produced correct outputs across multiple test conditions.

6. Black Box Testing

Black Box Testing was conducted without looking into the internal code, focusing only on input-output behavior. This type of testing simulated how an actual end-user would interact with the system.

Focus Areas:

- Fraud detection functionality.

- Error handling for invalid or incomplete transactions.
- User interface behavior for various inputs.

Result:

The system handled all test cases effectively, producing accurate predictions and user-friendly error messages.

7. Test Strategy and Approach

Testing was performed both manually and automatically using Python's unittest framework and Postman for API testing. Each test case was designed based on system requirements and real-world UPI transaction scenarios.

Test Objectives:

- All transaction inputs should be processed correctly.
- All dashboard pages and links must work properly.
- Predictions should be generated without delay.
- Fraud alerts must trigger automatically when probability exceeds the defined threshold.

Features to be Tested:

- Transaction data input validation.
- Prediction model output accuracy.
- Database record creation and update operations.
- Admin dashboard navigation.
- Alert notification system.

8. Integration Testing

Objective:

To verify that interconnected components — backend, ML model, and database — operate correctly as one integrated unit.

Steps Performed:

1. Sent API requests with sample transaction data.
2. Verified that the backend triggered the ML model correctly.
3. Confirmed that prediction results were stored in the database.
4. Checked that the dashboard displayed the correct results.

Result:

All test cases passed successfully, and no data loss or system crash occurred during integration.

9. Acceptance Testing

Acceptance Testing ensures that the developed system fulfills the end-user and business requirements and is ready for deployment.

Purpose:

To validate that the UPI Fraud Detection System meets its intended goal — accurately detecting and flagging suspicious transactions in real-time.

Performed By:

Bank Admins and QA Testers.

Test Results:

- The system correctly detected fraudulent transactions.
- No major defects were found during acceptance testing.
- User interface was intuitive and performed reliably.
- All acceptance criteria were met successfully.

RESULTS

The screenshot shows a web browser window with the URL `C:/Users/ADITYA2314/Desktop/codez/add.html`. The page title is "UPI Fraud Detect - Your Protect". The navigation bar includes "Home", "Login", "Sign Up", and "About". The main content area has a yellow background and displays the following information:

- MEDIUM Fraud Risk (47.78%)**
- Primary Model Used:** CNN Model
- CNN Confidence:** 61.83%
- Simple Model Confidence:** 56.82%

Analysis: The system detected a medium-risk transaction with a fraud probability of 47.78%.

Recommended Actions:

- Always verify the recipient's details before making a transaction.
- Be cautious of unsolicited requests for money or personal information.
- If the risk is HIGH, immediately contact your bank and report to cyber crime authorities.
- If the risk is MEDIUM, proceed with caution and double-check all details.
- If the risk is LOW, continue to be vigilant.

Analysis History:
Date: 18 Oct 2025, 00:03 (Sat)

The Windows taskbar at the bottom shows the time as 12:03 AM on 10/18/2025.

The screenshot shows the same web browser window, but the results are for a low-risk transaction. The main content area has a light green background and displays the following information:

- Primary Model Used:** CNN Model
- CNN Confidence:** 82.39%
- Simple Model Confidence:** 62.19%

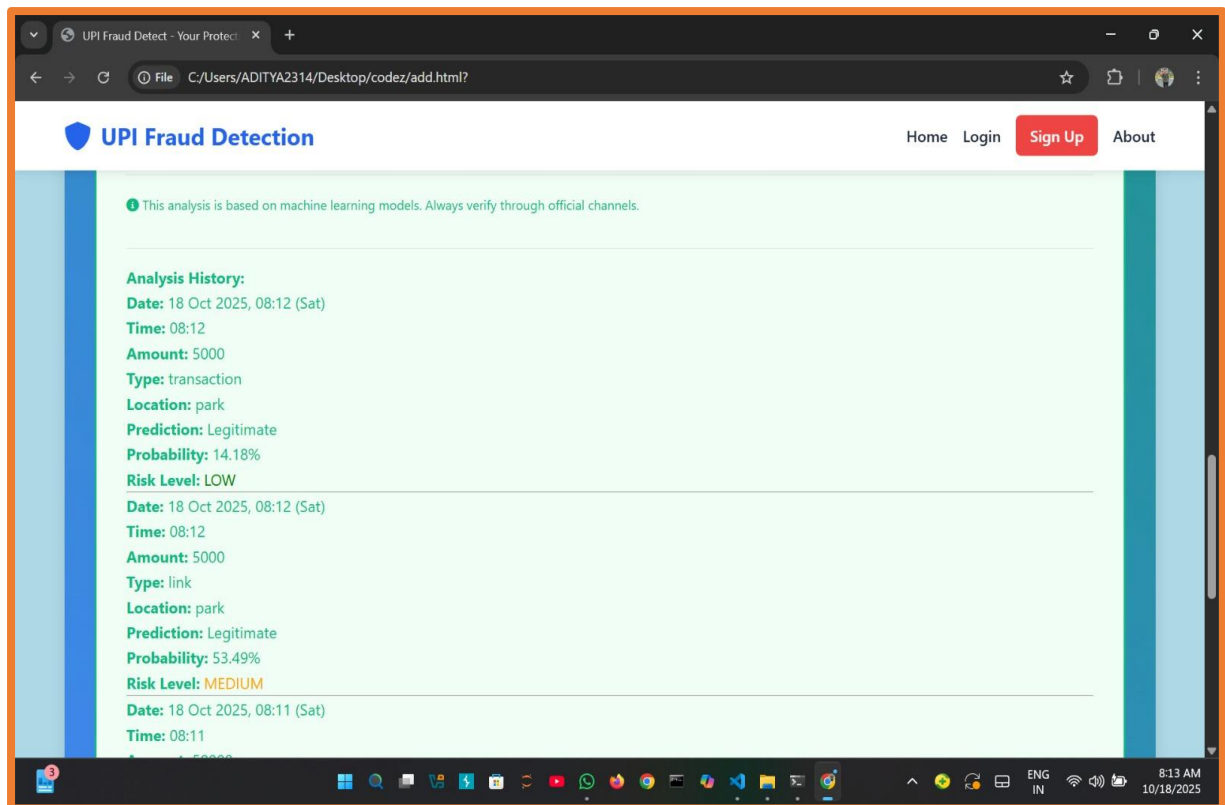
Analysis: The system detected a low-risk transaction with a fraud probability of 20.02%.

Recommended Actions:

- Always verify the recipient's details before making a transaction.
- Be cautious of unsolicited requests for money or personal information.
- If the risk is HIGH, immediately contact your bank and report to cyber crime authorities.
- If the risk is MEDIUM, proceed with caution and double-check all details.
- If the risk is LOW, continue to be vigilant.

Analysis History:
Date: 18 Oct 2025, 00:03 (Sat)
Time: 00:03
Amount: 500
Type: transaction
Location: unknown

The Windows taskbar at the bottom shows the time as 12:03 AM on 10/18/2025.



CONCLUSION

The UPI fraud detection system utilizing machine learning techniques, particularly the RandomForestClassifier, has proven to be a reliable and effective method for identifying fraudulent transactions. The model achieved strong performance metrics such as high precision and recall, indicating its capability to correctly detect fraud while minimizing false positives. Its ensemble nature enabled it to handle complex, nonlinear patterns within transaction data and provided resilience against overfitting.

By accurately detecting potentially fraudulent activities, this model can assist financial institutions in mitigating fraud risks, ensuring user safety, and maintaining trust in digital payment platforms. To further improve the system's performance and adaptability, continuous hyperparameter tuning and training on larger, real-world datasets are recommended. Incorporating additional behavioral, geolocation, and device-based features could further enhance detection capabilities, making the model a practical asset for real-time fraud prevention in the UPI ecosystem.

FUTURE ENHANCEMENTS

- **Real-Time Fraud Prediction Using Streaming Data:**

Enhance the system to process and analyze transactions in real-time using tools like Apache Kafka or Spark Streaming, allowing instant fraud detection as transactions occur.

- **Integration with UPI and Banking APIs:**

Connect directly with UPI gateways (like NPCI) and banking systems to automate the detection process and flag suspicious transactions without manual data uploads.

- **Advanced Deep Learning Models:**

Incorporate Recurrent Neural Networks (RNNs), LSTM, or Graph Neural Networks (GNNs) to capture complex temporal and relational patterns in fraudulent behavior for improved detection accuracy.

- **Federated Learning for Privacy Preservation:**

Enable decentralized model training using federated learning so that banks can collaboratively train fraud models without sharing sensitive customer data.

- **Explainable AI (XAI) Techniques:**

Add explainability modules (e.g., SHAP, LIME) to help analysts understand *why* a transaction was classified as fraudulent, improving transparency and trust.

- **Adaptive Fraud Scoring System:**

Develop a dynamic scoring mechanism that adjusts fraud risk thresholds automatically based on user behavior patterns and historical trends.

- **Blockchain Integration for Transaction Verification:**

Integrate blockchain technology to record and verify UPI transactions immutably, reducing the chances of data manipulation or false claims.

- **Automated Reporting & Alert System:**

Enhance the alerting mechanism with SMS, Email, and Dashboard notifications that include fraud type, probability score, and recommended actions for the user or admin.

- **Behavioral Biometric Authentication:**

Incorporate user behavior patterns (typing speed, touch dynamics, device motion) as additional features to authenticate users and detect imposters.

- **Cross-Bank Fraud Detection Network:**

Develop a centralized fraud detection network across multiple banks and payment service providers to share insights, detect linked fraudulent accounts, and prevent large-scale fraud.

REFERENCES

- [1] H. U. Sharif and M. A. Mohammed, "A literature review of financial losses statistics for cyber security and future trend," *IEEE Access*, vol. 15, pp. 138–156, 2022.
- [2] Y. Bao, G. Hilary, and B. Ke, "Artificial intelligence and fraud detection," in *Innovative Technology at the Interface of Finance and Operations (Springer Series in Supply Chain Management, Forth coming)*, vol. 1. Springer, 2022, pp. 223–247. [Online]. Available: <https://ssrn.com/abstract=3738618>
- [3] K.G.Al-Hashedi and P. Magalingam, "Financial fraud detection applying data mining techniques: A comprehensive review from 2009 to 2019," *IEEE Access*, vol. 40, 2021, Art. no. 100402.
- [4] F. Y. Osisanwo, J. E. T. Akinsola, O. Awodele, J. O. Hinmikaiye, O. Olakanmi, and J. Akinjobi, "Supervised machine learning algorithms: Classification and comparison," *Int. J. Comput. Trends Technol. (IJCTT)*, vol. 48, no. 3, pp. 128–138, 2017.
- [5] P. R. Vlachas, J. Pathak, B. R. Hunt, T. P. Sapsis, M. Girvan, E. Ott, and P. Koumoutsakos, "Backpropagation algorithms and reservoir computing in recurrent neural networks for the forecasting of complex spatiotemporal dynamics," *Neural Netw.*, vol. 126, pp. 191–217, Jun. 2020.
- [6] Varun Kumar K S, Vijaya Kumar V G, Vijay Shankar A, Pratibha K, "UPIFraud Detection using Machine Learning Algorithms", *International Journal of Engineering Research & Technology (IJERT)*, Volume 09, Issue 07 (July 2020), pp 5-8.
- [7] Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi and G. Bontempi, "UPIFraud Detection: A Realistic Modeling and a Novel Learning Strategy," in *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3784–3797, Aug. 2018.
- [8] Pratyush Sharma, Souradeep Banerjee, Devyanshi Tiwari, and Jagdish Chandra Patni, "Machine Learning Model for UPIFraud Detection-A Comparative Analysis", *The International Arab Journal of Information Technology*, Vol. 18, No. 6, November 2021, pp 789-790.
- [9] Ruttala Sailusha ; V. Gnaneswar ; R. Ramesh ; G. Ramakoteswara Rao , "UPIFraud Detection Using Machine Learning ", 2020 4th International Conference on Intelligent Computing and Control Systems (ICICCS), 19 June 2020
- [10] Kha Shing Lim, Lam Hong Lee and Yee-Wai Sim, "A Review of Machine Learning Algorithms for Fraud Detection in Credit Card Transaction", *IJCSNS International Journal of Computer Science and Network Security*, Vol.21 No.9, September 2021, pp 32-37.

